

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- EXPERIMENTS

Course Code:	CSA1209 & CSA1215	Course Title:	Computer Architecture
CO Assessed:	CO3	Assessment:	ASSESSMENT TOOL 1- EXPERIMENTS
Weightage:	40%	Total Marks:	25
CO3 Statement:	<i>Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT). (BL6)</i>		
Student Name:	Jannathul Firdaus N	Reg.No.:	192573025
Department:	CSE - DS	Date:	22.08.2026

ANNEXURE A

1. Design and simulate a system that performs register transfer operations along with basic arithmetic and logic operations such as addition, subtraction, AND, and OR. Use multiple registers to demonstrate how data is transferred between them during execution, and analyze the sequence of operations and the role of the ALU in processing data.

1. Aim

To design and simulate register transfer operations and basic arithmetic and logical operations using multiple registers and an ALU.

2. Objective

The system demonstrates:

- Register-to-register data transfer
- Addition
- Subtraction
- AND
- OR
- Role of ALU
- Sequence of operations

3. Python Implementation

Q1 - Register Transfer and ALU Simulation

class CPU:

```
def __init__(self):
```

```
    self.registers = {
```

```
        "R1": 0,
```

```
        "R2": 0,
```

```
        "R3": 0,
```

```
        "R4": 0
```

```
    }
```

```
def transfer(self, source, destination):
```

```
    self.registers[destination] = self.registers[source]
```

```
    print(f"Transfer: {source} -> {destination}")
```

```
    self.display()
```

```
def alu(self, operation, source1, source2, destination):
```

```
    a = self.registers[source1]
```

```
    b = self.registers[source2]
```

```
    if operation == "ADD":
```

```
        result = a + b
```

```
    elif operation == "SUB":
```

```
        result = a - b
```

```
    elif operation == "AND":
```

```
        result = a & b
```

```
    elif operation == "OR":
```

```
        result = a | b
```

```
    else:
```

```
        print("Invalid operation")
```

```
    return
```

```
self.registers[destination] = result
```

```
print(f"ALU: {source1} {operation} {source2} -> {destination}")
```

```
print(f"Calculation: {a} {operation} {b} = {result}")
```

```
self.display()
```

```
def display(self):
```

```
    print("Registers:", self.registers)
```

```
    print("-" * 40)
```

```
cpu = CPU()
```

```
print("INITIAL REGISTERS")
```

```
cpu.display()
```

```
# Load values
```

```
cpu.registers["R1"] = 20
```

```
cpu.registers["R2"] = 10
```

```
print("After loading R1=20 and R2=10")
```

```
cpu.display()
```

```
# Register transfer
```

```
cpu.transfer("R1", "R3")
```

```
# Arithmetic operations
```

```
cpu.alu("ADD", "R1", "R2", "R3")
```

```
cpu.alu("SUB", "R1", "R2", "R4")
```

Logical operations

cpu.alu("AND", "R1", "R2", "R3")

cpu.alu("OR", "R1", "R2", "R4")

Output

Output

INITIAL REGISTERS

Registers: {'R1': 0, 'R2': 0, 'R3': 0, 'R4': 0}

After loading R1=20 and R2=10

Registers: {'R1': 20, 'R2': 10, 'R3': 0, 'R4': 0}

Transfer: R1 -> R3

Registers: {'R1': 20, 'R2': 10, 'R3': 20, 'R4': 0}

ALU: R1 ADD R2 -> R3

Calculation: 20 ADD 10 = 30

Registers: {'R1': 20, 'R2': 10, 'R3': 30, 'R4': 0}

ALU: R1 SUB R2 -> R4

Calculation: 20 SUB 10 = 10

Registers: {'R1': 20, 'R2': 10, 'R3': 30, 'R4': 10}

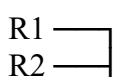
2. Create a model of a computer system using single-bus and multi-bus organization, and perform data transfer operations between registers, memory, and I/O. Compare the time taken and efficiency of both approaches, and analyze how multiple bus structures improve parallel data transfer and reduce bottlenecks.

1. Aim

To model single-bus and multi-bus computer organization and compare their data transfer efficiency.

Single-Bus Organization

In a single-bus system, all components share one common bus.





Only one major transfer can use the bus at a time.

Example

For:

$$R3 = R1 + R2$$

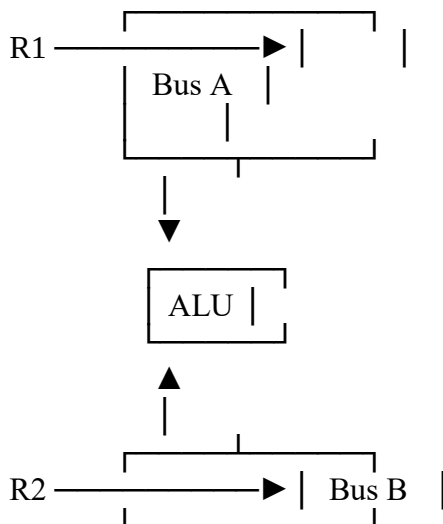
The processor needs:

$R1 \rightarrow \text{Bus}$
 $\text{Bus} \rightarrow \text{ALU}$
 $R2 \rightarrow \text{Bus}$
 $\text{Bus} \rightarrow \text{ALU}$
 $\text{ALU} \rightarrow R3$

This creates more transfer steps.

Multi-Bus Organization

A multi-bus system can transfer multiple operands simultaneously.



R1 and R2 can be supplied to the ALU at the same time.

Python Simulation

Q2 - Single Bus vs Multi Bus

```
def single_bus_operation():
```

```
    print("\nSINGLE BUS ORGANIZATION")
```

```
print("-" * 40)
```

```
cycles = 0
```

```
print("1. R1 -> Bus")
```

```
cycles += 1
```

```
print("2. Bus -> ALU")
```

```
cycles += 1
```

```
print("3. R2 -> Bus")
```

```
cycles += 1
```

```
print("4. Bus -> ALU")
```

```
cycles += 1
```

```
print("5. ALU -> R3")
```

```
cycles += 1
```

```
print("Total cycles:", cycles)
```

```
return cycles
```

```
def multi_bus_operation():
```

```
    print("\nMULTI BUS ORGANIZATION")
```

```
    print("-" * 40)
```

```
cycles = 0
```

```
print("1. R1 -> Bus A and R2 -> Bus B")
```

```
cycles += 1
```

```
print("2. ALU performs R1 + R2")
```

```
cycles += 1
```

```
print("3. ALU -> R3")
```

```
cycles += 1
```

```
print("Total cycles:", cycles)
```

```
return cycles
```

```
single = single_bus_operation()
```

```
multi = multi_bus_operation()
```

```
improvement = ((single - multi) / single) * 100
```

```
print("\nCOMPARISON")
```

```
print("-" * 40)
```

```
print("Single Bus cycles:", single)
```

```
print("Multi Bus cycles :", multi)
```

```
print(f"Efficiency improvement: {improvement:.2f}%")
```

Output

SINGLE BUS ORGANIZATION

```
-----  
1. R1 -> Bus  
2. Bus -> ALU  
3. R2 -> Bus  
4. Bus -> ALU  
5. ALU -> R3  
Total cycles: 5
```

MULTI BUS ORGANIZATION

```
-----  
1. R1 -> Bus A and R2 -> Bus B  
2. ALU performs R1 + R2  
3. ALU -> R3  
Total cycles: 3
```

3. Simulate a 5-stage instruction pipeline (Fetch, Decode, Execute, Memory, Write-back) and execute a set of instructions. Measure the performance in terms of speedup and throughput, and compare it with a non-pipelined system to evaluate the effectiveness of pipelining.

1. Aim

To simulate a 5-stage instruction pipeline and compare its performance with a non-pipelined processor.

2. Five Stages

IF → ID → EX → MEM → WB

Where:

Stage Meaning

IF Instruction Fetch

ID Instruction Decode

EX Execute

MEM Memory Access

WB Write Back

Pipeline Diagram

For five instructions:

Cycle →	1	2	3	4	5	6	7	8	9
I1	IF	ID	EX	MEM	WB				
I2		IF	ID	EX	MEM	WB			
I3			IF	ID	EX	MEM	WB		
I4				IF	ID	EX	MEM	WB	
I5					IF	ID	EX	MEM	WB

The important point is that **multiple instructions are being processed simultaneously in different stages**

Python Implementation

Q3 - Five Stage Pipeline Simulation

```
instructions = [  
    "I1: ADD R1,R2,R3",  
    "I2: SUB R4,R5,R6",  
    "I3: AND R7,R8,R9",  
    "I4: OR R10,R11,R12",  
    "I5: ADD R13,R14,R15"  
]  
  
stages = ["IF", "ID", "EX", "MEM", "WB"]  
  
n = len(instructions)  
  
print("5-STAGE PIPELINE")  
print("=" * 70)  
  
print("Cycle\t" + "\t".join(instructions))  
  
for cycle in range(1, n + len(stages)):  
    row = []  
  
    for i in range(n):  
        stage_index = cycle - i - 1  
  
        if 0 <= stage_index < len(stages):  
            row.append(stages[stage_index])  
        else:  
            row.append("-")  
  
    print(f"{cycle}\t" + "\t".join(row))
```

```
# Performance calculation
```

```
non_pipeline_cycles = n * len(stages)
```

```
pipeline_cycles = n + len(stages) - 1
```

```
speedup = non_pipeline_cycles / pipeline_cycles
```

```
throughput_non_pipeline = n / non_pipeline_cycles
```

```
throughput_pipeline = n / pipeline_cycles
```

```
print("\nPERFORMANCE")
```

```
print("=" * 40)
```

```
print("Instructions:", n)
```

```
print("Non-pipelined cycles:", non_pipeline_cycles)
```

```
print("Pipelined cycles:", pipeline_cycles)
```

```
print(f"Speedup: {speedup:.2f}x")
```

```
print(f"Non-pipelined throughput: {throughput_non_pipeline:.3f}")
```

```
print(f"Pipelined throughput: {throughput_pipeline:.3f}")
```

output

5-STAGE PIPELINE

=====					
=					
Cycle	I1: ADD R1,R2,R3		I2: SUB R4,R5,R6		I3: AND R7,R8,R9
	I4: OR R10,R11,R12		I5: ADD R13,R14,R15		
1	IF	-	-	-	-
2	ID	IF	-	-	-
3	EX	ID	IF	-	-
4	MEM	EX	ID	IF	-
5	WB	MEM	EX	ID	IF
6	-	WB	MEM	EX	ID
7	-	-	WB	MEM	EX
8	-	-	-	WB	MEM
9	-	-	-	-	WB

PERFORMANCE

=====

Instructions: 5
Non-pipelined cycles: 25
Pipelined cycles: 9
Speedup: 2.78x
Non-pipelined throughput: 0.200

Visualize

Run

4. Develop a pipeline execution scenario where data hazards, control hazards, and structural hazards occur. Observe their impact on instruction execution, and implement techniques such as stalling, data forwarding, and branch prediction to resolve these hazards and improve performance.

1. Aim

To simulate data, control and structural hazards and demonstrate techniques such as stalling, forwarding and branch prediction.

Types of Hazards

A. Data Hazard

Occurs when one instruction depends on the result of a previous instruction.

Example:

I1: ADD R1,R2,R3
I2: SUB R4,R1,R5

I2 needs R1, but I1 has not completed writing R1.

I1 → produces R1
↓
I2 → needs R1

B. Control Hazard

Occurs because of branch instructions.

Example:

I1: ADD R1,R2,R3
I2: BEQ R1,R0,LABEL
I3: SUB R4,R5,R6

The processor does not immediately know whether I3 should execute.

C. Structural Hazard

Occurs when two instructions need the same hardware resource simultaneously.

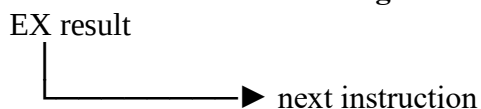
Example:

Instruction 1 → Memory
Instruction 2 → Memory

If only one memory unit exists, they compete for the resource.

Hazard Handling

Data hazard → Forwarding



Instead of waiting for the result to be written to the register.

Control hazard → Branch Prediction

Processor predicts:

Branch Taken

or:

Branch Not Taken

Structural hazard → Stalling

One instruction waits until the resource becomes available.

Python Simulation

Q4 - Pipeline Hazard Simulation

```

instructions = [

    ("I1", "ADD R1,R2,R3"),

    ("I2", "SUB R4,R1,R5"), # Data dependency

    ("I3", "BEQ R4,R0,L1"), # Control hazard

    ("I4", "LOAD R6,0(R7)", # Structural resource

    ("I5", "LOAD R8,4(R7)")

]

```

```

print("=" * 60)

print("PIPELINE HAZARD SIMULATION")

print("=" * 60)

```

```

# -----

# DATA HAZARD

# -----

```

```

print("\n1. DATA HAZARD")

print("-" * 40)

```

```

print("I1: ADD R1,R2,R3")

print("I2: SUB R4,R1,R5")

```

```

print("\nProblem:")

print("I2 needs R1 produced by I1.")

```

```

print("\nWithout forwarding:")

print("I2 must stall until R1 is available.")

```

```
print("\nWith forwarding:")  
  
print("Result from I1 EX stage is directly forwarded to I2.")
```

```
  
print("Stalls without forwarding: 2")  
print("Stalls with forwarding : 0")
```

```
  
# -----  
  
# CONTROL HAZARD  
  
# -----
```

```
  
print("\n2. CONTROL HAZARD")  
  
print("-" * 40)
```

```
  
print("I3: BEQ R4,R0,L1")
```

```
  
print("\nProblem:")  
  
print("Processor does not immediately know the branch result.")
```

```
  
print("\nWithout branch prediction:")  
  
print("Pipeline waits for branch decision.")
```

```
  
print("\nWith branch prediction:")  
  
print("Processor predicts branch direction.")
```

```
  
print("Penalty without prediction: 2 cycles")
```

```
print("Penalty with correct prediction: 0 cycles")
```

```
# -----
```

```
# STRUCTURAL HAZARD
```

```
# -----
```

```
print("\n3. STRUCTURAL HAZARD")
```

```
print("-" * 40)
```

```
print("I4: LOAD R6,0(R7)")
```

```
print("I5: LOAD R8,4(R7)")
```

```
print("\nProblem:")
```

```
print("Both instructions require the same memory resource.")
```

```
print("\nSolution:")
```

```
print("Processor stalls one instruction.")
```

```
print("Structural stall: 1 cycle")
```

```
# -----
```

```
# SUMMARY
```

```
# -----
```

```
print("\nSUMMARY")
```

```
print("=" * 60)
```

```
print("Data Hazard    -> Forwarding")
```

```
print("Control Hazard -> Branch Prediction")
```

```
print("Structural Hazard -> Stalling / Additional Hardware")
```

output

5. Analyze a processor supporting superscalar execution, including out-of-order and speculative execution, and extend the study to include hyper-threading and simultaneous multithreading (SMT). Evaluate how multiple instructions are executed in parallel and compare the performance improvements in terms of instruction throughput and resource utilization.

1. Aim

To simulate a superscalar processor supporting out-of-order execution, speculative execution and simultaneous multithreading.

2. Superscalar Processor

A normal scalar processor might execute:

1 instruction/cycle

A 2-wide superscalar processor can potentially execute:

2 instructions/cycle

Example:

Cycle 1:

ADD —————▶ ALU 1
SUB —————▶ ALU 2

3. Out-of-Order Execution

Suppose:

I1: LOAD R1
I2: ADD R2,R1,R3
I3: ADD R5,R6,R7

I2 must wait for I1.

But I3 does not depend on I1.

Therefore:

I1 → waiting

I2 → waiting

I3 → EXECUTE

This improves resource utilization.

4. Speculative Execution

For a branch:

BEQ R1,R2,LABEL

The processor predicts the branch.

If prediction is correct:

Continue execution

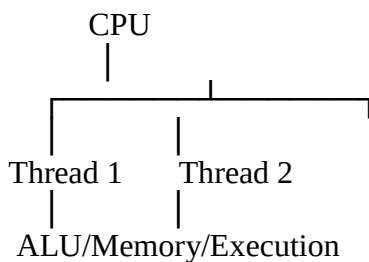
If prediction is wrong:

Discard speculative instructions

5. SMT

SMT allows multiple software threads to use the same processor resources.

For example:



Instead of leaving execution units idle when Thread 1 is waiting, Thread 2 can use them.

Python Simulation

Q5 - Superscalar, Out-of-Order, Speculative Execution and SMT

from collections import deque

```
instructions = [
    {"name": "I1", "type": "ALU", "depends": None},
    {"name": "I2", "type": "ALU", "depends": "I1"},
    {"name": "I3", "type": "ALU", "depends": None},
    {"name": "I4", "type": "MEM", "depends": None},
    {"name": "I5", "type": "ALU", "depends": "I4"},
```

```
]
```

```
completed = set()
waiting = instructions.copy()
```

```
cycle = 0
```

```
print("=" * 70)
print("SUPERSCALAR OUT-OF-ORDER SIMULATION")
print("=" * 70)
```

```
while waiting:
```

```
    cycle += 1
```

```
    print(f"\nCycle {cycle}")
```

```
    executed = []
```

```
    for instruction in waiting:
```

```
        dependency = instruction["depends"]
```

```
        if dependency is None or dependency in completed:
```

```
            print(f'Executing {instruction['name']} ({instruction['type']})')
```

```
            executed.append(instruction)
```

```
            break
```

```
    # Execute second independent instruction
```

```
    for instruction in waiting:
```

```
        if instruction in executed:
```

```
            continue
```

```
        dependency = instruction["depends"]
```

```
        if dependency is None or dependency in completed:
```

```
            print(f'Executing {instruction['name']} ({instruction['type']})')
```

```
            executed.append(instruction)
```

```
            break
```

```
    for instruction in executed:
```

```
        completed.add(instruction["name"])
```

```
        waiting.remove(instruction)
```

```
print("\nAll instructions completed.")
```

```
print("Total cycles:", cycle)
```

```
ideal_scalar_cycles = len(instructions)
```

```
speedup = ideal_scalar_cycles / cycle
```

```
print(f'Approximate superscalar speedup: {speedup:.2f}x')
```

```
# -----
```

```
# SPECULATIVE EXECUTION
```

```
# -----
```

```
print("\n" + "=" * 70)
```

```
print("SPECULATIVE EXECUTION")
```

```

print("=" * 70)

branch_prediction = "TAKEN"

print("Branch instruction detected.")
print("Processor predicts:", branch_prediction)

actual_result = "TAKEN"

if branch_prediction == actual_result:
    print("Prediction CORRECT.")
    print("Speculative instructions are committed.")
else:
    print("Prediction WRONG.")
    print("Speculative instructions are discarded.")

# -----
# SMT
# -----

print("\n" + "=" * 70)
print("SIMULTANEOUS MULTITHREADING")
print("=" * 70)

thread1 = deque([
    "T1-I1",
    "T1-I2",
    "T1-I3"
])

thread2 = deque([
    "T2-I1",
    "T2-I2",
    "T2-I3"
])

cycle = 0

while thread1 or thread2:

    cycle += 1

    print(f"\nCycle {cycle}")

    if thread1:
        instruction = thread1.popleft()
        print("Thread 1 executes:", instruction)

    if thread2:
        instruction = thread2.popleft()
        print("Thread 2 executes:", instruction)

print("\nBoth threads completed.")

```

Code of Conduct:

I Jannathul Firdaus N (192573025) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Understanding of Concepts (20%)	Experimental Design (20%)	Use of Tools (25%)	Analysis of Results (20%)	Documentation (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation